

Influence diagram files

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	Decisions and information arcs	2
4	Utilities	3
5	Checking the numbers	4
6	Multi-attribute utilities	5
7	Writing an influence diagram from Julia	7
8	Summary	8
9	References	8

1 Overview

Netica ([Norsys Software Corp. 2024](#)), GeNIe ([BayesFusion, LLC 2025](#)) and HUGIN ([HUGIN EXPERT A/S 2024](#)) store influence diagrams as well as Bayesian networks: chance nodes plus *decision* nodes, whose parents are information arcs, and *utility* nodes, whose parents are the variables the utility depends on. BIF, DSC and UAI have no such nodes and the writers refuse them with an `UnsupportedNodeError`.

This vignette reads the umbrella problem, which goes back to Raiffa ([1968](#)) and is the worked example of Shachter ([1986](#)), together with the SPEC section 46 grazing-management diagram, in the three formats, and shows how each element is represented.

2 Setup

Each fixture exists as .dne, .xdsl and .net, so the same diagram can be compared across the three influence-diagram formats.

```
using BayesianNetworkFormats

umbrella = Dict(ext => read_network(fixture_path("${ext}/umbrella.${ext}")) for ext
in ("dne", "xdsl", "net"))
umbrella["dne"]
```

```
NetworkIR("umbrella": 2 chance, 1 decision, 1 utility; format=:dne)
```

```
has_decisions(umbrella["dne"]), decisions(umbrella["dne"]),
utilities(umbrella["dne"])
```

```
(true, IRVariable[IRVariable(:Umbrella, decision, states=["take", "leave"],
parents=[:Forecast]), IRVariable[IRVariable(:U, utility, states=String[],
parents=[:Weather, :Umbrella])])
```

3 Decisions and information arcs

The decision Umbrella is informed by Forecast; that is the only parent it has, and it carries no table:

```
d = variable(umbrella["dne"], :Umbrella)
d.kind, d.states, d.parents, d.table
```

```
(DecisionNode, ["take", "leave"], [:Forecast], nothing)
```

Netica marks the node with kind = DECISION ([Norsys Software Corp. 2024](#)); GeNIe with a <decision> element ([BayesFusion, LLC 2025](#)); HUGIN with a decision block and an empty potential that lists the information parents ([HUGIN EXPERT A/S 2024](#)):

```
function show_block(name, pattern, n)
    lines = readlines(fixture_path(name))
    i = findfirst(l -> occursin(pattern, l), lines)
    println(join(lines[i:min(i + n - 1, length(lines))], "\n"))
end
show_block("dne/umbrella.dne", "node Umbrella", 6)
```

```
node Umbrella {
    kind = DECISION;
    discrete = TRUE;
    states = (take, leave);
    parents = (Forecast);
    title = "Take umbrella?";
```

```
show_block("xdsl/umbrella.xdsl", "<decision", 5)
```

```
<decision id="Umbrella">
  <state id="take"/>
  <state id="leave"/>
  <parents>Forecast</parents>
</decision>
```

```
show_block("net/umbrella.net", "potential (Umbrella", 3)
```

```
potential (Umbrella | Forecast)
{
}
```

4 Utilities

A utility node has no states. Its table has one axis per parent, in parent order, holding the utility of each parent configuration. $U(\text{Weather}, \text{Umbrella})$:

```
u = variable(umbrella["dne"], :U)
u.parents, size(u.table)
```

```
([:Weather, :Umbrella], (2, 2))
```

```
u.table # rows: sunny, rainy; columns: take, leave
```

```
2×2 Matrix{Float64}:
 20.0  100.0
 70.0   0.0
```

Netica keeps utilities in a functable laid out exactly like a probs block over the parents; GeNIe in `<utilities>`; HUGIN in the data of the utility's potential. All three are row-major with the last parent fastest:

```
show_block("dne/umbrella.dne", "node U {", 10)
```

```
node U {
  kind = UTILITY;
  discrete = FALSE;
  chance = DETERMIN;
  parents = (Weather, Umbrella);
  functable =
    // take  leave  // Weather
    ((20.0, 100.0), // sunny
     (70.0, 0.0)); // rainy
  title = "Satisfaction";
}
```

```
show_block("xdsl/umbrella.xdsl", "<utility", 4)
```

```
<utility id="U">
  <parents>Weather Umbrella</parents>
  <utilities>20.0 100.0 70.0 0.0</utilities>
</utility>
```

```
show_block("net/umbrella.net", "potential (U", 6)
```

```
potential (Umbrella | Forecast)
{
}
```

```
potential (U | Weather Umbrella)
{
}
```

5 Checking the numbers

The tables can be used directly. Without the forecast, the best fixed action has expected utility 70; with the forecast the optimal policy reaches 77, the values reported for this problem in the literature ([Shachter 1986](#)):

```
w = variable(umbrella["dne"], :Weather).table      # P(Weather)
f = variable(umbrella["dne"], :Forecast).table      # P(Forecast | Weather),
(weather, forecast)
eu_fixed = [sum(w[i] * u.table[i, a] for i in 1:2) for a in 1:2]
eu_no_info = maximum(eu_fixed)
eu_with_forecast = sum(maximum(sum(w[i] * f[i, k] * u.table[i, a] for i in 1:2) for
a in 1:2) for k in 1:3)
(no_information=eu_no_info, with_forecast=eu_with_forecast,
value_of_forecast=eu_with_forecast - eu_no_info)
```

```
(no_information = 70.0, with_forecast = 77.0, value_of_forecast = 7.0)
```

The three files agree with each other and with the IR written by hand in `test/reference_models.jl`:

```
[isequivalent(umbrella[a], umbrella[b]) for a in ("dne", "xdsl", "net"), b in
("dne", "xdsl", "net")]
```

```
3×3 Matrix{Bool}:
 1  1  1
 1  1  1
 1  1  1
```

6 Multi-attribute utilities

GeNIe can combine several utility nodes with a weighted `<mau>` node ([BayesFusion, LLC 2025](#)). Netica and HUGIN have no such node: they sum every utility node, which is a MAU with unit weights. The IR keeps MAU nodes in `ir.mau`; the Netica and HUGIN writers accept them only when all weights are one.

The grazing-management diagram of SPEC section 46 has two utilities, the conservation benefit of biodiversity and the cost of the management action, summed by `TotalUtility`:

```
grazing = Dict(ext =>
  read_network(fixture_path("${ext}/grazing_reference_id.${ext}")) for ext in ("dne",
  "xdsl", "net"))
grazing["xdsl"]
```

```
NetworkIR("grazing_reference_id": 9 chance, 1 decision, 2 utility; format=:xdsl)
```

```
grazing["xdsl"].mau
```

```
1-element Vector{MAUNode}:
 MAUNode{:TotalUtility, [:ConservationBenefit, :ManagementCost], [1.0, 1.0]}
```

```
show_block("xdsl/grazing_reference_id.xdsl", "<mau", 4)
```

```
<mau id="TotalUtility">
  <parents>ConservationBenefit ManagementCost</parents>
  <weights>1.0 1.0</weights>
</mau>
```

```
grazing["dne"].mau, grazing["net"].mau
```

```
(MAUNode[], MAUNode[])
```

The decision is informed by the climate forecast and the current vegetation survey, and acts on the ecosystem through an uncertain implementation, `GrazingPressure`:

```
gm = variable(grazing["dne"], :GrazingManagement)
gm.parents, gm.states
```

```
([:ClimateForecast, :CurrentVegetation], ["exclude", "reduce", "maintain"])
```

```
variable(grazing["dne"], :GrazingPressure).table # rows: exclude, reduce,
maintain; columns: low, high
```

```
3×2 Matrix{Float64}:
 0.95  0.05
 0.6   0.4
 0.15  0.85
```

```
variable(grazing["dne"], :ManagementCost).table
```

```
3-element Vector{Float64}:  
-40.0  
-15.0  
  0.0
```

Apart from the MAU node, which only GeNIe stores, the three files are equivalent:

```
[isequivalent(grazing[a], grazing[b]; mau=false, extras=false) for a in ("dne",  
"xdsl", "net"), b in ("dne", "xdsl", "net")]
```

```
3×3 Matrix{Bool}:  
 1  1  1  
 1  1  1  
 1  1  1
```

topological_order treats information arcs like any other arc, so decisions come after the variables they observe, and joint_distribution refuses a model with decisions, since there is no joint distribution until a policy is fixed:

```
topological_order(grazing["dne"])
```

```
12-element Vector{Symbol}:  
:Climate  
:CurrentVegetation  
:ClimateForecast  
:SoilMoisture  
:GrazingManagement  
:GrazingPressure  
:ManagementCost  
:Vegetation  
:HabitatQuality  
:Occupancy  
:Biodiversity  
:ConservationBenefit
```

```
try  
    joint_distribution(grazing["dne"])  
catch e  
    showerror(stdout, e)  
end
```

ValidationError: joint_distribution requires a network without decision nodes

7 Writing an influence diagram from Julia

An IR can be built directly and written in any of the three formats:

```
vars = [IRVariable(:Weather; states=["sunny", "rainy"], table=[0.7, 0.3]),
        IRVariable(:Forecast; states=["sunny", "cloudy", "rainy"],
        parents=[:Weather],
        table=[0.7 0.2 0.1; 0.15 0.25 0.6]),
        IRVariable(:Umbrella; kind=DecisionNode, states=["take", "leave"],
        parents=[:Forecast]),
        IRVariable(:U; kind=UtilityNode, parents=[:Weather, :Umbrella], table=[20.0
        100.0; 70.0 0.0])]
mine = NetworkIR("umbrella", vars)
dir = mktempdir()
write_network(joinpath(dir, "umbrella.net"), mine)
println(read(joinpath(dir, "umbrella.net"), String))
```

```
net
{
    node_size = (80 40);
    name = "umbrella";
}

node Weather
{
    states = ("sunny" "rainy");
}

node Forecast
{
    states = ("sunny" "cloudy" "rainy");
}

decision Umbrella
{
    states = ("take" "leave");
}

utility U
{
}

potential (Weather)
{
    data =
        (0.7 0.3);
}

potential (Forecast | Weather)
{
    data =
```

```

        ((0.7 0.2 0.1)                                % Weather=sunny
         (0.15 0.25 0.6));                             % Weather=rainy
    }

    potential (Umbrella | Forecast)
    {
    }

    potential (U | Weather Umbrella)
    {
        data =
            ((20.0 100.0)                             % Weather=sunny
             (70.0 0.0));                             % Weather=rainy
    }

```

```

try
    write_network(joinpath(dir, "umbrella.bif"), mine)
catch e
    showerror(stdout, e)
end

```

UnsupportedNodeError: node Umbrella of type decision is not supported by the bif format

8 Summary

Decision nodes carry information arcs and no table, utility nodes carry a table over their parents with no states of their own, and a multi-attribute utility node combines several utilities with weights; the three influence-diagram formats agree on all of it, and an IR built in Julia writes out to any of them. BIF, DSC and UAI raise `UnsupportedNodeError` instead. Evaluating such a diagram, computing optimal policies, expected utility and the value of information, is the job of `InfluenceDiagrams.jl`, which reads these same files through this package; for the probabilistic side, start again at *Reading and writing networks*.

9 References

- BayesFusion, LLC. 2025. *GeNIe Modeler and SMILE Engine Documentation*. <https://support.bayesfusion.com/docs/>.
- HUGIN EXPERT A/S. 2024. *HUGIN API Reference Manual: The NET Language*. <https://download.hugin.com/webdocs/manuals/api-manual.pdf>.
- Norsys Software Corp. 2024. *Netica File Formats*. https://www.norsys.com/WebHelp/NETICA/X_File_Formats.htm.

Raiffa, Howard. 1968. *Decision Analysis: Introductory Lectures on Choices Under Uncertainty*. Addison-Wesley.

Shachter, Ross D. 1986. "Evaluating Influence Diagrams." *Operations Research* 34 (6): 871–82.
<https://doi.org/10.1287/opre.34.6.871>.